

# BLIS 如何表示异构集群：现状、机制与做异构研究需要改的东西

ours 分支（上游 2ebef6a），2026-09-26

## 1 结论摘要

1. 我们目前的仿真是同构集群。每台实例（instance，一个运行 vLLM 引擎的推理副本）都是 Qwen3-14B、H100、张量并行度 1（TP，一个副本占一块 GPU）、同样的 KV 块数、同样的步时长模型。系统规模  $n$  就是  $n$  台一模一样的单 GPU 服务器。
2. BLIS 有”节点池”（node pools）机制来声明多种 GPU 各多少台，但它只改变三样东西：每台实例的 KV 容量、最大上下文长度、每小时成本。步时长（一步计算要多久）不会随 GPU 类型改变：代码里预留了”按 GPU 覆盖延迟系数”的字段，但命令行和策略文件都没有地方填它（上游 issue #893 承认此事）。所以今天用节点池混合 H100 与 L40S，L40S 只是”内存小的 H100”。
3. 路由看不见 GPU 类型。路由快照里有 GPU 类型和成本字段，但所有评分器只读队列长度和 KV 利用率（比例）。一台 4 千块的 L40S 和一台 1.6 万块的 H100 在利用率 50% 时被视为相同。
4. PD 分离（prefill 与 decode 分池）是 BLIS 里唯一”按角色”的异构，可按角色设 TP、KV 容量、上下文上限、延迟后端，但同样不能按角色设 GPU 速度。
5. 我们的共享队列模式在异构下能跑但有隐含偏向：唤醒最低编号实例、fcfs-pool 不分实例、Mooncake 用 0 号实例的步时长模型、后加入的实例拿不到共享队列。
6. 做异构研究的最小改动是一处：在 cmd/root.go 里为每个节点池加载对应 GPU 的硬件校准并填入 HWConfigByGPU。此外要放弃 --total-kv-blocks（否则所有实例同容量），并决定唤醒规则与集合策略是否应感知实例速度。

## 2 我们现在跑的是什么

ours/sweep.py 的基本参数：`--model qwen/qwen3-14b --hardware H100 --tp 1 --num-instances n --total-kv-blocks B`，加共享队列（默认）或 push 路由（B1 系列），不传 `--policy-config`。因此：没有节点池，没有放置，没有成本，没有拓扑；所有实例用同一套 H100 校准（trained-physics 后端，用 H100 的峰值算力 989.5 TFLOPS 和显存带宽 3.35 TB/s 做基函数，系数在 defaults.yaml 里只有一套，是在 H100 轨迹上拟合的）；每台实例  $B$  块（15,909 或 6,500），由命令行显式给定，而不是按显存自动算。

### 3 BLIS 里的异构机制

#### 3.1 节点池：如何声明”每种 GPU 各几台”

在 `--policy-config` 的 YAML 里写 `node_pools` (类型 `NodePoolConfig`, `sim/cluster/infra_config.go`)。字段：

| 字段                                | 含义                                                                      |
|-----------------------------------|-------------------------------------------------------------------------|
| <code>name</code>                 | 池名，必须唯一                                                                 |
| <code>gpu_type</code>             | GPU 类型字符串，必须跨池唯一（成本与容量按它查）                                              |
| <code>gpus_per_node</code>        | 每节点 GPU 数 ( $\geq 1$ )                                                  |
| <code>gpu_memory_gib</code>       | 每块 GPU 显存，用于自动算 KV 容量                                                   |
| <code>initial_nodes</code>        | 起始节点数（就是”服务器数”）                                                         |
| <code>min_nodes, max_nodes</code> | 上下界： <code>max_nodes</code> 必须 $\geq$ <code>initial_nodes</code> ，所以必须写 |
| <code>provisioning_delay</code>   | 新节点上线延迟（在 <code>blis run</code> 里没有代码会新建节点，等于无用）                        |
| <code>cost_per_hour</code>        | 每节点每小时成本                                                                |

一个最小例子（4 台 H100、8 台 L40S，每台一块 GPU， $TP = 1$ ）：

`node_pools:`

- {name: h100, gpu\_type: H100, gpus\_per\_node: 1, gpu\_memory\_gib: 80, initial\_nodes: 4, max\_nodes: 4, cost\_per\_hour: 4.0}
- {name: l40s, gpu\_type: L40S, gpus\_per\_node: 1, gpu\_memory\_gib: 48, initial\_nodes: 8, max\_nodes: 8, cost\_per\_hour: 1.2}

`instance_lifecycle:`

`warm_start_initial_instances: true`

运行时仍要 `--num-instances 12`:实例数来自命令行,不是池的总和。`gpu_memory_gib` 与 `hardware_config.json` 里的 `MemoryGiB` 是两处独立的数，要手工保持一致。

#### 3.2 放置：哪台实例落到哪个池

`PlaceInstance` (`sim/cluster/infra_placement.go`) 按实例编号 0, 1, 2, ...依次放置，第一遍在”单节点首次适配”下按池的**声明顺序**、节点编号、空闲 GPU 顺序找位置；第二遍只在  $TP$  大于每节点 GPU 数时跨整节点放置。含义：

- 先声明的池先被填满。上例里实例 0 到 3 落在 H100，4 到 11 落在 L40S。
- 没有”把实例  $i$  钉在池  $j$ ”的办法，唯一的控制是池的顺序和数量。
- 实例数超过总容量时，多出的实例进入 `pending`，只打一条警告，此后永远不会被放置 (`NodeReadyEvent` 在 `blis run` 里没有生产代码触发)，相当于被静默丢弃。

### 3.3 放置之后每台实例得到什么

三处放置代码（启动、延迟放置、自动扩容）做同样的几步：

1. **GPU 标签**：simCfg.GPU 设为池的 gpu\_type。只是标签。
2. **延迟系数（不生效）**：代码为 `if hc, ok := config.HWConfigByGPU[type]; ok { simCfg.HWConfig = hc }`。HWConfigByGPU 在 DeploymentConfig 里有定义，但 cmd/ 从未赋值，策略文件也没有对应的键（sim/cluster/network\_topology.go 的注释明说：mixed-gpu\_type fleet currently shares one calibration）。因此每台实例都用 `--hardware` 那一种 GPU 的校准算步时长。
3. **KV 容量**（applyPerInstanceKVCapacity, sim/cluster/kv\_autocalc.go）：仅当没有显式传 `--total-kv-blocks` 时，用池的 gpu\_memory\_gib 重算：

$$\text{blocks} = \frac{\text{显存} \times \text{利用率} \times TP - (\text{权重} + \text{激活 } 5.5 \text{ GiB} + \text{非 torch 开销})}{16 \text{ tokens} \times \text{每 token KV 字节}}$$

用同一 fork 二进制实测 Qwen3-14B、TP 1、利用率 0.9：H100 与 A100-80 都是 15,909 块，L40S 4,113 块。显式给了 `--total-kv-blocks` 则所有实例同容量。

4. **上下文上限**：若 max-model-len 超过该实例的 KV 容量，就按容量封顶（小 GPU 的实例会拒绝更长的请求）。
5. **成本**：InstanceCostPerHour 等于池的每小时成本乘以跨越的节点数。它不进任何输出指标，只供自动扩缩容和路由快照使用。

### 3.4 步时长如何依赖 GPU（trained-physics 后端）

步时长 = 一组 roofline 基函数（按 token 数、上下文长度、层数算出的计算量和访存量，分别除以 GPU 的峰值算力 TFlopsPeak 和显存带宽 BwPeakTBs）乘以一套拟合系数  $\alpha, \beta$ 。GPU 类型只通过这两个峰值（以及权重为 1 字节时的 TFlopsFP8，多节点时的互连带宽比）进入。随附的 hardware\_config.json 只有四个条目：

| GPU                | 峰值 TFLOPS (bf16) | 显存带宽 TB/s | 显存 GiB |
|--------------------|------------------|-----------|--------|
| H100               | 989.5            | 3.35      | 80     |
| A100-SXM / A100-80 | 312              | 2.04      | 80     |
| L40S               | 362              | 0.86      | 48     |

系数  $\alpha, \beta$  全局只有一套，在 H100 上拟合；defaults.yaml 里 30 个模型条目全部标 H100。所以 A100 或 L40S 的绝对延迟是“用 H100 系数按峰值等比缩放”的结果，没有校准过。作为研究里的“快机器与慢机器”是可用的，但不要把它们绝对数字当成 A100 或 L40S 的实测。

### 3.5 路由能否区分快慢实例

路由快照 RoutingSnapshot 携带 GPUType、TPDegree、CostPerHour、TotalKvCapacityTokens、KvTokensInUse、MaxBatchSize（在 cluster\_event.go 填入）。但没有任何评分器读它们：

| 评分器                               | 读取的量            |
|-----------------------------------|-----------------|
| queue-depth, load-aware           | 队列长度            |
| kv-utilization                    | 1- 已用块 / 总块（比例） |
| active-requests, running-requests | 在途数、批大小         |
| vllm-dp                           | 4× 队列长度 + 批大小   |

快照里的 TTFT、ITL、DispatchRate 字段从未被生产代码赋值，恒为 0。因此今天的路由策略只能通过积压间接感知快慢；要做异构路由需要新的评分器（读 GPU 类型、绝对容量或步时长）。

### 3.6 PD 分离：按角色的异构

--prefill-instances p --decode-instances d (另有共享角色) 按编号分配角色：先 prefill、再 decode、再共享。每角色可覆盖 tp、hardware、latency-model、max-model-len、MoE 通信后端 (ResolvePoolConfig, sim/cluster/resolve.go)。其中 --prefill-hardware L40S 同样只改标签和该角色的 KV 自动计算，不改延迟校准；能真正进入步时长的角色差异只有 TP 和延迟后端。KV 传输成本 = 基础延迟 0.05 ms + 传输字节 / 25 GB/s (可开争用：带宽按并发传输数均分)，与 GPU 类型和放置无关。注意 PD 与节点池同时用时放置不分角色（角色按编号，池按顺序）。

### 3.7 自动扩缩容与成本

生产接线是 DefaultCollector → V2SaturationAnalyzer → UnlimitedEngine → DirectActuator。分析器按 (GPU 类型, TP) 分组比较“KV 供给 = 总 KV token × 阈值”与“KV 需求 = 在用 token + 队列长度 × 平均输入”；引擎扩最便宜的变体、缩最贵的。它只在现有节点的空闲 GPU 上放实例，从不新建节点；没有延迟或速度模型。成本没有任何输出指标，要自己按每小时成本之和乘以时长来算。

### 3.8 限制

| 组合                  | 结果                   | 位置                     |
|---------------------|----------------------|------------------------|
| blis replay + 节点池   | 致命错误                 | cmd/replay.go          |
| blis observe        | 无 --policy-config 参数 |                        |
| MoE --dp > 1 + 节点池  | 致命错误                 | cmd/root.go            |
| 角色 TP ≠ 全局 TP + 节点池 | panic                | sim/cluster/cluster.go |
| 重复 gpu_type 或池名     | panic                | infra_placement.go     |

## 4 我们的共享队列模式在异构下的问题

1. 命令行检查只拒绝 --flow-control 和 prefill/decode 分池，不检查 --prefill-decode-instances；节点池可以直接与共享队列同用。

2. **唤醒规则**：到达和完成都唤醒”最低编号的空闲实例”，而放置先填先声明的池，所以 YAML 里池的顺序变成了 GPU 类型偏向：轻载时工作总是先给第一个池。
3. **fcfs-pool 不知道是谁在取**：BatchContext 没有实例身份或速度字段。快实例更频繁到达步边界，隐含地多拉工作，这是”自然的”，但任何想按速度分配的集合策略都需要新增字段。
4. **按实例的上下文上限丢弃**：AdoptSharedRequest 若请求超过本机的 MaxModelLen 或 KV 容量就丢弃。小 GPU 实例会丢掉大 GPU 本可服务的请求，且与自检报告 A2 的”先分配再取件”问题叠加。
5. **Mooncake 用 0 号实例的步时长模型**（cluster.go，注释写着 all instances share one config），异构下预测是错的。alpha 延迟取 0 号实例无妨，因为该延迟是全局常数。
6. **后加入的实例拿不到共享队列**：SetSharedQueue 只在启动时调用，自动扩容或延迟上线的实例永远不会从共享队列取件。异构研究若只用启动时的固定实例集则无影响。
7. **实例加载延迟非零时**，InstanceLoadedEvent 不唤醒共享队列；用 warm\_start\_initial\_instances: true 或零延迟。

## 5 做异构研究需要改什么

1. **按 GPU 的步时长（唯一的硬阻塞）**：在 cmd/root.go 组装 DeploymentConfig 处，对每个节点池调用 latency.GetHWConfig(hwConfigPath, pool.gpu\_type) 填入 HWConfigByGPU。放置代码三处已经会读它。约 20 行加一个测试。这一步之后，L40S 实例才会真的慢（带宽 0.86 对 3.35 TB/s，解码步长约 3 到 4 倍）。
2. **KV 容量**：去掉 --total-kv-blocks 让自动计算生效（并按池设 gpu\_memory\_gib）；或者新增”按池显式块数”（目前没有）。注意我们现在的 6,500 块远低于 H100 的物理容量 15,909，若要保持”内存受限”的研究设定，需要按池给显式块数，这一项也要加。
3. **放置控制**：按想要的顺序声明池，--num-instances 恰好等于总容量；多余实例被静默丢弃。
4. **共享队列**：唤醒规则改为轮转或按速度；给 SetPolicy 的上下文加实例身份、容量、步时长；AdoptSharedRequest 的按机丢弃要改成”放回队列给别的实例”或在入队时按最大实例检查；Mooncake 改为每实例的步时长函数；addLiveInstance 补 SetSharedQueue。
5. **路由基线**：现有评分器都是比例或计数，需要一个感知容量或速度的评分器作为 push 模式的异构基线。
6. **校准诚实**：非 H100 的绝对延迟未校准，报告里把它们写成”慢 GPU 变体（峰值等比缩放）”。
7. **成本**：没有成本输出，要在 analyze.py 里按池成本乘以窗口长度补一列，目标函数里才有”每小时成本”项。
8. **回放不可用**：节点池只在 blis run 里有效。

## 6 建议（供你决定，不会自行执行）

若下一步研究“快慢混合的 GPU 池里如何分配请求”，最小可行形态是：两个池（H100 与 L40S，或 H100 与 A100-80 作为“同显存但慢 3 倍”的干净对照），启动时固定实例数，先做第 1 项改动并用单实例验证 L40S 的步时长确实变慢，再决定共享队列的唤醒规则和集合策略如何感知实例速度。这里的每一项选择（池的比例、显式块数、唤醒规则、集合策略）都是研究判断，需要你来定。